

Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет ИТМО»

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ «Экспериментальное введение в операционные системы»

Исследование механизмов файлового ввода-вывода: read/lseek vs mmap
Вариант: Read, Cache vs No Cache, 7M, Rand

Выполнил: студент группы Р3332
Гузалов Тимур Павлович
Преподаватель: Тюрин Иван Николаевич

Санкт-Петербург
2026

Содержание

1. Введение и цель работы	3
2. Паспорт системы и фиксация окружения	4
Настройка тестового окружения (Environment Hardening)	4
3. Подготовка данных и формулировка гипотез	5
Генерация графа	5
Формулировка гипотез	5
4. Методология сбора и статистической обработки	6
5. Результаты измерений и анализ	7
Сводная статистика времени выполнения (Wall Clock Time)	7
Анализ системных метрик ядра ОС	7
Детальный разбор феномена 28.24 с vs 0.045 с (Физика ввода-вывода)	7
6. Исследование интрузивности инструментов мониторинга	9
7. Ответы на контрольные вопросы защиты	10
8. Заключение	11

1. Введение и цель работы

Цель работы: Экспериментально исследовать и сравнить эффективность двух фундаментальных механизмов доступа к файлам в операционных системах Linux — традиционного интерфейса системных вызовов (`read/lseek`) и отображения файлов в виртуальную память (`mmap`) — при обходе случайных графовых структур данных в режимах работы с системным кэшем страниц (Page Cache) и в условиях его обхода/сброса.

Параметры выданного варианта:

- **Тип нагрузки (Workload):** Read (холостой обход графа без модификации вершин);
- **Фактор сравнения (Comparison axis):** Cache vs No Cache (сравнение горячего кэша и принудительного холодного доступа);
- **Размер тестовых данных (Size):** 7М (7,340,032 байта);
- **Топология графа (Topology):** Rand (случайное расположение соседних узлов в файле со скачками между страницами);
- **Зерно генератора (Seed):** 435169.

В качестве модели данных используется связный список (цепной граф), сериализованный в бинарный файл. Каждая вершина имеет размер 24 байта, заголовок файла — 40 байт. Параметр `--min-step-pages 2.0` гарантирует, что следующий переходит минимум через 2 страницы (8192 байта), тем самым нивелируя внутривстраничную пространственную локальность и нагружая подсистему подкачки памяти.

2. Паспорт системы и фиксация окружения

Для воспроизводимости эксперимента перед измерениями был снят паспорт аппаратного и программного обеспечения тестового стенда.

Параметр	Значение / Характеристика
Процессор (CPU)	AMD Ryzen (Zen 4 + Zen 4c), 10 ядер / 20 потоков
Архитектура ядер	4 P-ядра (до 5.09 ГГц) + 6 E-ядер (до 3.32 ГГц)
Кэш процессора	L1d/L1i: [48K/32K], L2: [1M/ядро], L3: [24M]
Оперативная память	32 ГБ (31369.3 MiB) LPDDR5/DDR5, частота 8000МГц
Накопитель (Disk)	NVMe HP SSD FX900 Pro 2TB
Файловая система	Btrfs поверх блочного шифрования dm-crypt/LUKS
Операционная система	Linux (дистрибутив NixOS), glibc 2.42
Версия ядра (Kernel)	Linux 7.2.2
Инициализация / Окружение	KDE Plasma, Wayland, фоновые фоновые демоны минимизированы

Настройка тестового окружения (Environment Hardening)

Для снижения дисперсии (джиттера) и систематической ошибки таймингов были применены следующие меры:

1. Отключение динамического разгона (Turbo Boost):

```
echo 0 | sudo tee /sys/devices/system/cpu/cpufreq/boost
```

В результате частота всех ядер была жестко зафиксирована на базовой отметке 2.00 ГГц, что предотвратило температурный троттлинг.

- Фиксация регулятора частоты (CPU Governor):** установлен профиль performance через TLP.
- Изоляция процесса на конкретном ядре:** во всех тестах задействовалось производительное ядро №2 (P-core):

```
taskset -c 2 <binary>
```

- Приоритет планировщика:** критичные серии запускались под `chrt -f 99` (real-time FIFO), исключая вытеснение процесса фоновыми задачами ядра.
- Методология очистки кэша (No Cache):** Для имитации первого холодного запуска и проверки дисковой подсистемы перед каждым «posache»-прогоном выполнялась синхронизация и полный сброс дисковых буферов ядра:

```
sync && echo 3 | sudo tee /proc/sys/vm/drop_caches
```

3. Подготовка данных и формулировка гипотез

Генерация графа Генерация бинарного графа выполнена скриптом `graphgen.py`:

```
./graphgen.py -s 7M --seed 435169 --topology chain -b 0.5 --min-step-pages 2 -o graph-rand.bin
```

Свойства файла `graph-rand.bin`:

- Общий размер: 7,340,032 байт (7.00 МиБ);
- Число вершин (`node_count`): 305,833;
- Размер записи (`record_size`): 24 байта;
- Минимальный шаг прыжка: 342 вершины (8208 байт, 2 страницы по 4096 байт);
- Файл записан в один непрерывный экстенд Btrfs (1 extent found), фрагментация отсутствует.

Формулировка гипотез

1. **Гипотеза 1 (`read/lseek`):** При горячем кэше (`cache`) время будет определяться накладными расходами на переключение контекста между `user-space` и `kernel-space` при системных вызовах (305,833 вызовов `lseek` и `read`). Без кэша (`nocache`) произойдет колоссальная деградация производительности, так как каждый случайный переход потребует синхронного блокирующего ввода-вывода (`I/O latency bound`), усугубляемого накладными расходами шифрования `dm-crypt/LUKS`.
2. **Гипотеза 2 (`mmap`):** Отображение файла в память покажет существенно лучшее время в режиме `cache` из-за полного отсутствия системных вызовов в цикле обхода (доступ через разыменовывание указателей). В условном режиме `nocache` (сброс страниц) ожидается деградация из-за обработки страничных прерываний (`Page Faults`), которая приведёт к близким к `nocache read()` результатам.

4. Методология сбора и статистической обработки

Эксперимент состоит из четырёх конфигураций:

1. `read/lseek (cache)` — прогретый Page Cache;
2. `read/lseek (nocache)` — сброс кэша (`drop_caches`) перед замером;
3. `mmap (cache)` — прогретый Page Cache;
4. `mmap (nocache)` — сброс кэша (`drop_caches`).

Фиксация метрик производилась через утилиту `/usr/bin/time -v`. Первые 3 прогона каждой серии исключались из выборки (`warmup`).

Оценка статистической значимости: Для каждого параметра рассчитывались: выборочное среднее $\bar{x}$, выборочное стандартное отклонение s , стандартная ошибка среднего $SEM = \frac{s}{\sqrt{N}}$, а также границы 95%-го доверительного интервала по распределению Стьюдента:

$$\Delta = t_{0.975, N-1} \cdot SEM, \quad CI = [\bar{x} - \Delta, \bar{x} + \Delta]$$

Относительная ошибка оценивалась как $Rel_Error = \left(\frac{\Delta}{\bar{x}}\right) \cdot 100\%$.

5. Результаты измерений и анализ

Сводная статистика времени выполнения (Wall Clock Time)

В таблице ниже представлены агрегированные результаты серий измерений.

Метод	Режим	N	Mean [s]	Std [s]	CI 95% [s]	Rel. Err
mmap	cache	40	0.01875	0.00607	±0.00194	10.36%
mmap	nocache	40	0.04500	0.00679	±0.00217	4.83%
read/lseek	cache	150	0.15967	0.00316	±0.00051	0.32%
read/lseek	nocache	150	28.24400	0.34560	±0.05576	0.20%

Рис. 1: Графики сравнения времени выполнения (логарифмическая и линейная шкалы) сохранены в `stage3_final_time_comparison.png`.

Анализ системных метрик ядра ОС

Метод	Режим	sys [s]	user [s]	vol_ctx	invol	minflt	majflt
mmap	cache	0.0000	0.0165	1	1.3	341	0.0
mmap	nocache	0.0005	0.0160	75	2.0	340	10.1
read/lseek	cache	0.1218	0.0314	1	1.2	229	0.0
read/lseek	nocache	1.6213	0.2202	307 030	10.5	232	0.0

Детальный разбор феномена 28.24 с vs 0.045 с (Физика ввода-вывода)

Наиболее драматический результат эксперимента — колоссальный разрыв во времени холодного выполнения между `read/lseek` (28.24 с) и `mmap` (0.045 с), что составляет разницу в 627 раз.

1. **Поведение `read/lseek --no-cache`:** Программа выполняет обход 305, 833 вершин. В режиме случайного графа каждая вершина находится на другой странице. Системный вызов `read()` без кэша транслируется ядром в синхронные прямые запросы к диску. Процесс вынужден засыпать в ожидании ответа от NVMe контроллера на каждой итерации. Метрика `vol_ctx` = 307 030 свидетельствует о том, что процесс добровольно отдавал процессор планировщику 307 тысяч раз. Средняя задержка одного случайного запроса составила:

$$t_{IO} = \frac{28.24 \text{ с}}{305833} \approx 92.3 \text{ мкс}$$

Это вполне соответствует времени произвольного доступа блочного 4K-чтения современного NVMe SSD с учетом криптографического слоя LUKS.

2. **Поведение `mmap --no-cache`:** В операционной системе Linux системный вызов `mmap()` **резонно не поддерживает синхронный прямой ввод-вывод (`O_DIRECT`)**. Отображение всегда обслуживается через Page Cache посредством механизма страничных сбоев (Page Faults). Потому отсутствие кэширования мы имплементируем сбросом кэшей ОС.

Анализ событий ядра с помощью `perf record -e major-faults,block:block_rq_issue` показал:

```
graph_traverse_ [002]: major-faults: __memmove_avx512_unaligned_erms
graph_traverse_ [002]: block:block_rq_issue: 259,1 RA 131072 () + 256
```

```
graph_traverse_ [002]: block:block_rq_issue: 259,1 RA 131072 ( ) + 256  
... (всего 9-10 запросов)  
graph_traverse_ [002]: block:block_rq_issue: 259,1 RA 4096 ( ) + 8
```

Вывод ядра: При возникновении первого же серьезного страничного сбоя (major-fault) подсистема памяти Linux активировала механизм упреждающего чтения (**Readahead**). Ядро запросило с накопителя файл блоками размером по **128 КиБ** на линейной скорости шины PCIe. Файл размером 7 МБ был подгружен в физическую память всего за **10 мажорных сбоев** (majflt = 10.1), после чего дальнейший обход происходил чисто в ОЗУ без единого системного вызова и блокировки потока (volctx = 75).

6. Исследование интрузивности инструментов мониторинга

Для оценки влияния измерительных средств на чистоту эксперимента были сопоставлены замеры одной конфигурации (`read/lseek` `cache`) под разными инструментами:

Инструмент	Wall Time [s]	Замедление	Интрузивность
Базовый запуск (<code>time</code>)	0.159 с	1.00x	Нулевая
Аппаратные события (<code>perf stat</code>)	0.165 с	1.03x	Низкая
Трассировка (<code>strace -c</code>)	3.182 с	20.01x	Неприятная

Вывод: Аппаратные счетчики производительности PMU (`perf stat`) используют прерывания таймера ядра и спецрегистры CPU, практически не влияя на ход выполнения (+3% накладных расходов). В то же время перехват вызовов через `ptrace` (`strace`) накладывает двойное переключение контекста на каждый из 600,000 вызовов `read/lseek`, искажая временную картину на порядок. Для финальных серий использовался только неинвазивный `/usr/bin/time`.

7. Ответы на контрольные вопросы защиты

1. Во сколько раз отличаются показатели и подтвердилась ли гипотеза?

- read/lseek: отключение кэша замедляет обход в **176.8 раз** (0.159 с против 28.24 с).
- mmap: отключение кэша замедляет обход всего в **2.4 раза** (0.018 с против 0.045 с) засчёт политики readahead.
- В горячем кэше mmap быстрее read/lseek в **8.5 раз** за счет исключения накладных расходов на системные вызовы.
- Гипотеза подтвердилась с важным уточнением: предсказанная катастрофическая деградация без кэша коснулась только read/lseek, тогда как mmap был спасен упреждающей подгрузкой данных.

2. Оценка доверительных интервалов и достаточности выборки N :

- Для read/lseek (nocache) относительная ошибка составляет 0.20%, интервалы сравниваемых групп не пересекаются ($p < 0.001$).
- Для mmap (cache) относительная ошибка составила 10.36%. Это объясняется дискретностью системного таймера ядра: при среднем времени 18 мс флуктуации в 1–2 мс вносят заметный вклад. По формуле $N \geq \left(\frac{t \cdot s}{\varepsilon \cdot x}\right)^2$ для достижения ошибки 3% выборку mmap необходимо увеличить до $N \approx 180$.

3. Обоснование размера графа (7М) относительно RAM: Объем ОЗУ стенда составляет 32 ГБ, что более чем на 3 порядка превышает размер графа (7 МБ). Это гарантирует, что файл целиком помещается в Page Cache и кэш процессора. Следовательно, в режиме cache исследовалась чистая скорость работы с памятью без вытеснения страниц.

8. Заключение

1. Механизм `mpar` демонстрирует абсолютное превосходство над интерфейсом `read/lseek` при произвольном доступе к данным как в условиях кэширования (быстрее в 8.5 раз), так и при холодном старте (быстрее в 627 раз).
2. Системные вызовы `read()` в сценарии без кэша приводят к сериализации случайного ввода-вывода: SSD вынужден обрабатывать единичные блоки 4К, простаивая по шине данных и упираясь в физическую задержку переключения ячеек флеш-памяти.
3. Отображение `mpar` нивелирует случайный характер обхода за счет `Readahead` подсистемы виртуальной памяти, преобразуя случайные обращения к адресам памяти в последовательное чтение файла крупными блоками по 128 КиБ.
4. Меры по фиксации окружения (отключение `Turbo Boost`, привязка к Р-ядру через `taskset`, вытеснение через `chrt`) оказались ключевыми для снижения дисперсии измерений на микро-секундных интервалах.